

Ascent 1.0 — Master Source Index + Build Continuity Spine

Purpose of This Master Source File

This document is the **single source reference point** for Ascent 1.0 moving forward.

It exists because the source file limit has been reached, and the project can no longer depend on keeping every individual build prompt uploaded at the same time.

Instead of carrying 25+ separate files, this master file preserves the active logic, build history, rules, source hierarchy, and prompt continuity needed to keep Ascent 1.0 moving forward without losing context.

This is the living spine.

1. Master Source Index + Build Continuity Spine

1.1 Product Name

Ascent 1.0

All future references must use this naming convention.

Do not refer to the project as Asset 1.0 unless correcting older language.

1.2 Product North Star

Ascent 1.0 is a responsive web-based maintenance intelligence and continuous improvement operating system.

Its purpose is to help property managers, maintenance supervisors, regional maintenance leaders, and operations teams understand:

1. **What is happening now**
2. **Where the bottleneck is**
3. **What is causing operational drag**
4. **What records prove it**
5. **What needs action next**
6. **What patterns are forming over time**

Ascent 1.0 is not just a dashboard.

It is a connected operating system that turns messy operational data into **visibility, proof, priority, and action**.

1.3 Core System Identity

Ascent 1.0 must operate as a three-layer system:

A. Input Layer

This is how data enters the system.

Includes:

1. CSV uploads
2. Work order reports
3. Turn / make-ready records
4. Preventative maintenance records
5. Asset records
6. Warranty records
7. Documents
8. Photos
9. Manual updates
10. Assignment corrections
11. Future voice notes or structured field updates

B. Logic Layer

This is where the system interprets the data.

Includes:

1. Assignment engine
2. Dual-mode import governance
3. Reporting ingestion layer
4. Workflow engine
5. Work order logic
6. Turn logic
7. Scoring + stoplight engine
8. Alert + warning engine
9. Dashboard intelligence layer
10. Evidence/document logic
11. Reaction layer
12. Reporting + analytics spine
13. Future PM intelligence

14. Future asset/warranty intelligence
15. Future recalculation engines

C. Output Layer

This is what users see and act on.

Includes:

1. Control Tower dashboard
2. Operational Health tile
3. Work Order Performance tile
4. Turn Performance tile
5. PM Performance tile
6. Asset Performance tile
7. Priority Actions panel
8. Drill-down panels
9. Unit profiles
10. Property views
11. Reports area
12. Supporting record tables
13. Data quality and reporting readiness views

Input feeds logic. Logic feeds visibility. Visibility drives action.

1.4 Current Active Product Direction

The current product direction is centered around the **Control Tower** as the customer-facing operational source of truth.

The Control Tower should show five top-level tiles:

1. **Operational Health**
2. **Work Order Performance**
3. **Turn Performance**
4. **PM Performance**
5. **Asset Performance**

Below those tiles:

1. **Priority Actions Panel**

The Control Tower must use real industry language.

It should not overwhelm users with internal system terms.

A customer should understand the dashboard in under five seconds.

Health → Priority → Bottleneck → Context → Action.

1.5 Current System Truth

Ascent 1.0 has been built through several connected layers.

The system already includes, or is expected to preserve:

1. Authentication
2. Role-based access
3. App shell / navigation
4. Organization structure
5. Property structure
6. Unit intake
7. Unit profiles
8. Assignment engine
9. Workflow engine
10. Work order logic
11. Turn logic
12. Scoring + stoplight engine
13. Alert + warning engine
14. Dashboard intelligence layer
15. Document + evidence engine
16. Evidence visibility layer
17. Signal strength + language layer
18. System integration + reaction layer
19. Dual-mode import governance
20. Build 7 reporting spine
21. Build 7.1 reporting ingestion layer
22. Control Tower dashboard pre-wiring
23. Universal wiring audit prompt
24. Clickable signal verification prompt

Future builds must **extend** these systems.

They must not rebuild, duplicate, or replace them unless specifically directed.

2. Archived Prior Build Spine

This section preserves the role of each prior source file so older PDFs can be removed after their logic is safely captured here.

2.1 Continuous Improvement App Core Builds

Role

Original system blueprint and product North Star.

Preserved Logic

Ascent 1.0 must function as a continuous improvement operating system, not a disconnected dashboard.

The system must:

1. Show operational status using red / yellow / green logic
2. Track workflows and bottlenecks
3. Capture documents and evidence in context
4. Recalculate impact when conditions change
5. Track assets and warranties
6. Support both quick visibility and deep reporting
7. Work first as a responsive web app
8. Use one shared backend and database

Status

Active authority logic preserved in this Master Spine.

2.2 Phase 1 — Build 1: Foundation Prompt

Role

Authentication, roles, app shell, database foundation, and protected routes.

Preserved Logic

Build 1 created the base layer.

It established:

1. Email/password authentication
2. Sign up
3. Login
4. Logout
5. Protected routes
6. Role-based access
7. Organization structure

8. User profiles
9. Settings foundation
10. Navigation shell
11. Dashboard placeholder
12. App-wide route structure

Future Rule

Do not rebuild authentication or app shell unless explicitly directed.

Status

Archived into Master Spine.

2.3 Phase 1 — Build 1.5: Guided System Activation + Unit Intake Layer

Role

Customer onboarding, property setup, unit intake, and CSV/manual unit entry.

Preserved Logic

Build 1.5 created the system's first operational source of truth.

It established:

1. Guided setup after signup
2. Organization confirmation
3. Property creation
4. Unit intake choice
5. CSV upload path
6. Manual unit entry path
7. Unit records tied to organization and property
8. Units page foundation
9. Duplicate prevention
10. Persistent onboarding completion state

Future Rule

Unit and property data must remain part of one shared source of truth.

Do not create a separate intake system.

Status

Archived into Master Spine.

2.4 Phase 1 — Build 1.6: Unit Profile + Immediate History View

Role

Unit-level visibility and history context.

Preserved Logic

Build 1.6 made each unit a real operational record.

It established:

1. Clickable unit rows
2. Dedicated unit profile route
3. Unit header
4. Related work count
5. Related document count
6. Related asset count
7. Latest activity
8. Unit history timeline
9. Unit evidence section
10. Unit work/issues section
11. Unit asset section
12. Simple insight strip

Future Rule

Unit profiles must read from shared data.

No fake unit summaries.

No shadow history.

Status

Archived into Master Spine.

2.5 Phase 1 — Build 1.7: Assignment Engine Logic

Role

Matching incoming operational records to the correct property, unit, asset, workflow, or workflow item.

Preserved Logic

Build 1.7 established the assignment engine.

It supports:

1. Structured input records
2. Field normalization
3. Property matching
4. Unit matching
5. Asset matching where available
6. Confidence levels
7. High-confidence auto-assignment
8. Medium-confidence confirmation flow
9. Low-confidence review queue
10. Unit profile integration
11. Workflow integration where applicable
12. Document integration using the existing document engine
13. Manual reassignment
14. Audit history

Future Rule

Never auto-invent links.

Only high-confidence matches may auto-assign.

Medium and low confidence must be reviewed or confirmed.

Status

Archived into Master Spine.

2.6 Phase 1 — Build 1.8: System Integration + Reaction Layer

Role

Converts signals into clickable, provable data.

Preserved Logic

Build 1.8 established the system principle:

SIGNAL → CLICK → DATA → DECISION

Every major signal must become actionable.

A signal must be able to open supporting records showing:

1. Exact records creating the signal
2. Property
3. Unit
4. Status
5. Category
6. Stage
7. Time or aging logic
8. Explanation of why the record was included

Future Rule

No signal should appear unless it can be verified.

If supporting data is not available, the signal must not pretend to be clickable.

Status

Active authority logic preserved in this Master Spine.

2.7 Phase 1 — Build 2: Core Workflow Engine

Role

Workflow backbone and item movement.

Preserved Logic

Build 2 created the operational workflow foundation.

It established:

1. Workflows
2. Workflow stages
3. Workflow items
4. Workflow item history

5. Workflow assignments
6. Comments/notes
7. Stage movement
8. Timing foundation
9. Bottleneck foundation
10. Workflow list page
11. Workflow detail page
12. Item creation
13. Item movement
14. Stage counts
15. Basic aging and bottleneck indicators

Future Rule

Workflow logic must not be replaced.

Future work order, turn, PM, and asset logic must connect into or alongside the workflow structure as appropriate.

Status

Archived into Master Spine.

2.8 Phase 1 — Build 3: Scoring + Stoplight Engine

Role

Shared scoring brain.

Preserved Logic

Build 3 created shared score logic.

It established:

1. Flow calculator
2. Risk calculator
3. Improvement calculator
4. Execution calculator
5. Workflow health scoring
6. Operational health scoring
7. Shared stoplight threshold system
8. Real data scoring rules
9. Dashboard scoring outputs
10. Workflow scoring outputs
11. Critical item logic

12. Trend foundation
13. Explainability text
14. Low-data states

Future Rule

No dashboard, report, alert, or tile should create separate scoring logic if a shared scoring engine already exists.

Status

Archived into Master Spine.

2.9 Phase 1 — Build 4: Alert and Warning Engine

Role

Operational risk detection and warning system.

Preserved Logic

Build 4 established alert logic.

It supports:

1. Status alerts
2. Date/timing alerts
3. Flow alerts
4. Risk alerts
5. Informational alerts
6. Warning alerts
7. Critical alerts
8. Deduplication
9. Alert lifecycle
10. Active alerts
11. Alert history
12. Dashboard integration
13. Workflow integration
14. Acknowledge/resolve behavior
15. Scoring engine integration

Future Rule

Alerts must be generated from real rules and real data.

No fake alert activity.

No duplicate spam alerts.

Status

Archived into Master Spine.

2.10 Phase 1 — Build 5: Dashboard Intelligence Layer

Role

Turned the dashboard into a control-tower-style intelligence surface.

Preserved Logic

Build 5 established:

1. Executive snapshot
2. Flow / Risk / Improvement / Execution cards
3. Action panel
4. Bottleneck intelligence
5. Visual flow section
6. Workflow spotlight
7. Lightweight trend signals
8. Critical item engine
9. Role-aware dashboard
10. Explainability
11. Empty/low-data states

Future Rule

Dashboard intelligence must be powered by shared engines.

No static metrics.

No duplicate calculations inside UI components.

Status

Archived into Master Spine.

2.11 Phase 1 — Build 6: Document + Evidence Engine

Role

Document upload, attachment, preview, and evidence context.

Preserved Logic

Build 6 established:

1. Document upload
2. Photo/image upload
3. PDF upload
4. Attachment to workflow items
5. Attachment to workflows
6. Document metadata
7. In-flow document preview
8. Attachment count
9. Document type tagging
10. Document history
11. Audit log support
12. Role-based access

Future Rule

Documents must live in context.

They must not become a detached file bucket.

Status

Archived into Master Spine.

2.12 Phase 1 — Build 6.5: Evidence Visibility Layer

Role

Made evidence visible across decision surfaces.

Preserved Logic

Build 6.5 established:

1. Global attachment signal model
2. Attachment counts

3. Paperclip indicators
4. Evidence summary inside item detail panels
5. Priority action evidence badges
6. Bottleneck evidence context
7. Workflow list evidence signals
8. Dashboard critical missing documentation count
9. Alert hook for critical items missing documents

Future Rule

Evidence signals should be visible without forcing the user to dig.

Status

Archived into Master Spine.

2.13 Phase 1 — Build 6.6: Signal Strength + Language Layer

Role

Standardized evidence language.

Preserved Logic

Build 6.6 established shared language for evidence states:

1. **No documents**
2.  **Missing documentation**
3.  **{count}**
4.  **{count} docs**

It also established reusable components:

1. EvidenceBadge
2. EvidenceWarningLabel
3. AttachmentCountBadge

Future Rule

Do not hardcode evidence text inside components.

Use centralized language rules.

Status

Archived into Master Spine.

2.14 System Sweep / Demo Reset Prompt

Role

Clean demo/test data without breaking the app.

Preserved Logic

The system sweep prompt clears non-essential data while preserving:

1. Authentication
2. Users
3. Organizations
4. Roles
5. Permissions
6. Properties and units structure
7. Database schema
8. Routing
9. App shell
10. Workflow engine
11. Scoring engine
12. Alert engine
13. Document/evidence engine
14. Assignment engine
15. System integration engine

Future Rule

Reset the data.

Do not delete the machine.

Status

Archived into Master Spine.

2.15 Dual-Mode Import Governance Prompt

Role

Protects upload flexibility and truth-layer integrity.

Preserved Logic

The governance model has two modes:

Mode 1 — User Ingestion Mode

Flexible customer-facing upload mode.

Purpose:

1. Accept imperfect data
2. Preserve momentum
3. Classify records honestly
4. Route incomplete records to review
5. Prevent unresolved records from corrupting truth layers

Mode 2 — System Validation Mode

Strict internal validation mode.

Purpose:

1. Wiring audits
2. Stress tests
3. Demo validation
4. Pre-release verification
5. Architecture integrity checks

Resolution States

Every imported row must be classified as:

1. **Fully Resolved**
2. **Partially Resolved**
3. **Unresolved**

Future Rule

Users may upload messy data.

The system may not treat messy data as fully resolved truth.

Status

Active authority logic preserved in this Master Spine.

2.16 Phase 1 — Build 1.11.5: Control Tower Dashboard Restructure

Role

Simplified the dashboard into a true Control Tower.

Preserved Logic

Build 1.11.5 shifted the dashboard from too many signals to a focused control system:

1. Health
2. Priority
3. Bottleneck
4. Context

It replaced scattered action cards with a stronger **Priority Actions** model.

Priority Actions should focus on:

1. Work Orders
2. Turns
3. Preventative Maintenance

Each action must include:

1. Category label
2. Bold issue summary
3. Why it matters
4. Click interaction
5. Supporting records

Future Rule

The dashboard must direct attention, not just display information.

Status

Active authority logic preserved in this Master Spine.

2.17 Universal Wiring Audit Prompt

Role

Full system truth test.

Preserved Logic

This prompt asks:

Is the system actually connected, or just looking connected?

It validates:

1. Input validation
2. Assignment engine
3. Context layer
4. Workflow engine
5. Scoring engine
6. Alert engine
7. Dashboard intelligence
8. Reaction layer
9. End-to-end record traceability

The full trace must be:

INPUT → ASSIGNMENT → CONTEXT → WORKFLOW → SCORING → ALERT → DASHBOARD → CLICK → DATA

Future Rule

If the system proves full wiring, return **VERIFIED**.

If any layer fails, return **NOT WIRED**.

No soft language.

Status

Active authority logic preserved in this Master Spine.

2.18 Clickable Signal Verification Prompt

Role

Reaction layer proof test.

Preserved Logic

This prompt verifies:

SIGNAL → CLICK → SUPPORTING RECORDS → DECISION

Every clickable element must return:

1. Correct records
2. Matching count
3. Property context
4. Unit context
5. Category context
6. Status context
7. Stage context
8. Filter alignment

Failure Conditions

Mark FAIL if:

1. Click returns zero results when signal is greater than zero
2. Returned count does not match signal count
3. Records do not match signal criteria
4. Context is missing
5. Click does nothing
6. Click returns unrelated data

Future Rule

A number on screen means nothing until the click proves the records behind it.

Status

Active authority logic preserved in this Master Spine.

2.19 Build 7 — Master Spine: Reporting + Analytics Backbone

Role

Created reporting architecture.

Preserved Logic

Build 7 established the reporting spine.

Reporting must follow:

REAL DATA → SHARED LOGIC → REPORT STRUCTURE → EXPLAINABLE OUTPUT

The reporting spine supports:

1. Operational reports
2. Workflow reports
3. Evidence/documentation reports
4. Assignment/data quality reports
5. Asset/PM future-ready reports
6. Date range filters
7. Summary metrics
8. Trend foundation
9. Bottleneck analysis
10. Timing analysis
11. Risk analysis
12. Evidence analysis
13. Assignment coverage analysis
14. Narrative insight foundation
15. Report definitions registry
16. Drill-down compatibility
17. Reports page foundation
18. Snapshot support where needed

Future Rule

Reports must be explainable and tied to supporting records.

No fabricated trends.

No fake reporting summaries.

Status

Active authority logic preserved in this Master Spine.

2.20 Ascent 1.12.5 Control Tower Pre-Wiring

Role

Current 5-tile Control Tower layout direction.

Preserved Logic

This source defines the top-level dashboard as:

1. Operational Health
2. Work Order Performance
3. Turn Performance

4. PM Performance
5. Asset Performance
6. Priority Actions Panel

Each tile must include:

1. Title
2. Stoplight color
3. Score or key metric
4. Click interaction
5. Drill-down panel below the tiles
6. Supporting records
7. Unit/property reference

Multiple panels may stay open at once.

Future Rule

The Control Tower is the customer-facing validation source.

It must remain connected to the same source of truth as the Reports area.

Status

Active authority logic preserved in this Master Spine.

2.21 Build 7.1 — Reporting Ingestion Layer

Role

Trusted reporting intake gate.

Preserved Logic

Build 7.1 created the layer that asks:

Can this data be trusted enough to report?

It established:

1. Reporting ingestion architecture
2. Reporting source registry
3. Normalized reporting record contract
4. Reporting eligibility classifications
5. Dual-mode governance integration
6. Ingestion summary output

7. Reporting readiness panel
8. Control Tower compatibility
9. Work order reporting ingestion
10. Turn reporting ingestion
11. PM reporting ingestion readiness
12. Asset + warranty reporting ingestion readiness
13. Document + evidence reporting ingestion
14. Assignment coverage reporting ingestion
15. Drill-down compatibility
16. Reporting ingestion audit log
17. Role-aware reporting ingestion
18. Honest empty/low-data states

Reporting Eligibility States

Every normalized record must be classified as:

1. **Fully Reportable**
2. **Partially Reportable**
3. **Not Reportable Yet**

Future Rule

No Control Tower or Reports metric may silently mix fully reportable, partially reportable, and unresolved records as if they are equal.

Status

Active authority logic preserved in this Master Spine.

3. Active Authority Files to Keep Uploaded

These are the highest-value files to keep active as source references when upload slots are limited.

If file space is tight, these should be prioritized over older standalone build prompts.

3.1 Keep Active: Continuous Improvement App Core Builds

Why

This is the original blueprint and North Star.

It defines the product as an operating system, not just a dashboard.

Active Use

Use when confirming:

1. Overall app purpose
 2. Input / logic / output structure
 3. Core build direction
 4. Continuous improvement alignment
 5. Asset, warranty, workflow, dashboard, alert, document, and reporting relationships
-

3.2 Keep Active: Build 7 Master Spine

Why

This defines the reporting backbone.

Active Use

Use when creating or auditing:

1. Report structures
 2. Reporting logic
 3. Summary metrics
 4. Trend foundations
 5. Report definitions
 6. Analysis blocks
 7. Narrative insight rules
 8. Reports page behavior
-

3.3 Keep Active: Build 7.1 Reporting Ingestion Layer

Why

This is the trusted reporting intake gate.

Active Use

Use when creating or auditing:

1. Reporting ingestion
2. Normalized reporting records
3. Fully reportable vs partially reportable logic
4. Not reportable yet logic
5. Reporting readiness summaries

6. Control Tower compatibility
 7. Work order / turn / PM / asset reporting ingestion
 8. Data quality reporting
-

3.4 Keep Active: Dual-Mode Import Governance

Why

This protects the system from treating incomplete uploads as complete truth.

Active Use

Use when creating or auditing:

1. Upload behavior
 2. CSV parsing
 3. Flexible customer ingestion
 4. Strict internal validation
 5. Resolution states
 6. Import summaries
 7. Review queues
 8. Assignment confidence
 9. Downstream dashboard/reporting eligibility
-

3.5 Keep Active: Universal Wiring Audit Prompt

Why

This is the full-system truth test.

Active Use

Use after every major build to confirm:

1. Data enters correctly
 2. Data assigns correctly
 3. Data appears in context
 4. Workflows function
 5. Scores reflect real data
 6. Alerts trigger correctly
 7. Dashboard matches real state
 8. Clicks show supporting records
-

3.6 Keep Active: Clickable Signal Verification Prompt

Why

This tests whether the reaction layer is truly wired.

Active Use

Use after dashboard, work order, turn, report, or Control Tower updates to confirm:

1. Every signal is clickable
 2. Every click returns correct records
 3. Counts match source data
 4. Filters align
 5. Drill-downs preserve context
 6. No dead counts exist
-

3.7 Keep Active: Control Tower Pre-Wiring / Current Dashboard Direction

Why

This preserves the current customer-facing dashboard direction.

Active Use

Use when building or testing:

1. Operational Health tile
 2. Work Order Performance tile
 3. Turn Performance tile
 4. PM Performance tile
 5. Asset Performance tile
 6. Priority Actions panel
 7. Drill-down panels
 8. Multi-panel dashboard expansion
 9. Industry-language dashboard experience
-

4. Files That Can Be Removed After Archiving

The following files can be removed from active source uploads once this Master Spine is saved and uploaded.

Their logic has been captured in this document.

4.1 Safe to Archive / Remove from Active Uploads

1. Phase 1 Build 1 — Foundation Prompt
 2. Phase 1 Build 1.5 — Guided System Activation + Unit Intake
 3. Phase 1 Build 1.6 — Unit Profile + Immediate History View
 4. Phase 1 Build 1.7 — Assignment Engine Logic
 5. Phase 1 Build 2 — Workflow Engine
 6. Phase 1 Build 3 — Scoring + Stoplight Engine
 7. Phase 1 Build 4 — Alert and Warning Engine
 8. Phase 1 Build 5 — Dashboard Intelligence Layer
 9. Phase 1 Build 6 — Document + Evidence Engine
 10. Phase 1 Build 6.5 — Evidence Visibility Layer
 11. Phase 1 Build 6.6 — Signal Strength + Language Layer
 12. System Sweep / Demo Reset Prompt
 13. Phase 1 Build 1.11.5 — Control Tower Dashboard Restructure
-

4.2 Important Note

Removing older files does not mean their logic is forgotten.

Their core system requirements are now preserved in this Master Spine.

The Master Spine becomes the reference point.

The project stops depending on scattered files and starts depending on one living source.

5. Source Memory Header for Future Replit Prompts

Use this header at the top of every future Replit Agent prompt unless a specific prompt requires a different intro.

5.1 Standard Source Memory Header

SOURCE CONTINUITY NOTE

This build continues Ascent 1.0 from the Master Source Index + Build Continuity Spine.

The current system already includes:

- authentication and role-based access
- app shell / navigation shell
- organization, property, and unit structure
- guided system activation and unit intake
- unit profile and immediate history visibility
- assignment engine logic
- dual-mode import governance
- workflow engine
- work order logic
- turn logic
- timing and bottleneck foundation
- scoring + stoplight engine
- alert and warning engine
- dashboard intelligence layer
- document + evidence engine
- evidence visibility layer
- signal strength + language layer
- system integration + reaction layer
- Build 7 reporting spine
- Build 7.1 reporting ingestion layer
- Control Tower dashboard pre-wiring
- universal wiring audit logic
- clickable signal verification logic

Do NOT rebuild prior systems.

Do NOT redesign the application unless specifically instructed.

Do NOT duplicate scoring, alert, dashboard, assignment, document, work order, turn, reporting, or Control Tower logic.

Do NOT create fake dashboard or reporting results.

Do NOT treat partial data as complete truth.

Do NOT create dead-end counts, fake signals, or unclickable proof claims.

All new work must connect to the shared source of truth.

All surfaced metrics must be explainable.

All clickable signals must return supporting records.

All reporting outputs must distinguish confirmed truth from partial or unresolved data.

Continue from the existing architecture.

Extend the system.

Do not restart it.

5.2 Short Source Memory Header

Use this shorter version when the Replit prompt is already long.

SOURCE CONTINUITY NOTE

Continue Ascent 1.0 from the Master Source Index + Build Continuity Spine.

The system already includes auth, roles, app shell, properties, units, assignment, workflow, work orders, turns, scoring, alerts, dashboard intelligence, documents/evidence, reaction layer, import governance, Build 7 reporting spine, Build 7.1 reporting ingestion, and Control Tower pre-wiring.

Do NOT rebuild prior systems.

Do NOT duplicate existing logic.

Do NOT invent fake data, fake metrics, or fake wiring.

Do NOT treat partial data as complete truth.

All new work must connect to the shared source of truth and preserve SIGNAL → CLICK → SUPPORTING RECORDS → DECISION.

6. Ongoing Source Management Rule

This section explains how source files should be handled moving forward.

6.1 The New Rule

Do not keep adding separate source files forever.

After every major build, update this Master Spine instead of creating a new permanent standalone source file.

One living spine. Fewer loose files. Stronger continuity.

6.2 Build Update Process

After each major build, add a new section to this document with:

1. Build number
2. Build name

3. Purpose
 4. What was added
 5. What was changed
 6. What was removed
 7. What existing systems it depends on
 8. What new rules it created
 9. What must be tested next
 10. Whether the build is active authority or archived into the spine
-

6.3 Build Summary Template

Use this template after every major build.

```
## Build [Number] – [Build Name]

### Role

[Explain what this build does.]

### Systems It Extends

1. [Existing system]
2. [Existing system]
3. [Existing system]

### What This Build Added

1. [Feature / logic]
2. [Feature / logic]
3. [Feature / logic]

### What This Build Changed

1. [Change]
2. [Change]
3. [Change]

### What This Build Removed

1. [Removed item]
2. [Removed item]
3. [Removed item]

### New Rules Created
```

1. [Rule]
2. [Rule]
3. [Rule]

Required Validation

1. [Test]
2. [Test]
3. [Test]

Status

Active authority logic preserved in Master Spine.

6.4 Source Manifest Rule

Maintain a separate simple file called:

Ascent 1.0 Source Manifest

This can live outside the source upload area.

It should track every source file ever created.

Recommended fields:

1. File name
2. Build number
3. Purpose
4. Active or archived
5. Date archived into Master Spine
6. Whether it still needs to remain uploaded
7. Notes

6.5 Source Manifest Template

```
# Ascent 1.0 Source Manifest
```

```
## [File Name]
```

```
Build Number:
```

```
Purpose:
```

Status: Active / Archived
Archived into Master Spine: Yes / No
Still needs to remain uploaded: Yes / No
Notes:

6.6 Active Source Slot Strategy

When source slots are limited, prioritize files in this order:

1. Master Source Index + Build Continuity Spine
2. Continuous Improvement App Core Builds
3. Build 7 Master Spine
4. Build 7.1 Reporting Ingestion Layer
5. Dual-Mode Import Governance
6. Universal Wiring Audit Prompt
7. Clickable Signal Verification Prompt
8. Control Tower Pre-Wiring / Current Dashboard Direction
9. Most recent active build prompt
10. Most recent build result or audit output

Older build prompts should only stay uploaded if they are directly needed for a current audit or rebuild.

7. Non-Negotiable System Rules

These rules must guide all future prompts and audits.

7.1 No Rebuild Rule

Do not rebuild existing systems unless explicitly directed.

Future prompts should extend the current system, not restart it.

7.2 One Source of Truth Rule

Ascent 1.0 must use one shared truth layer.

Do not create duplicate data models for:

1. Units
2. Properties

3. Work orders
 4. Turns
 5. Documents
 6. Assignments
 7. Reports
 8. Dashboard metrics
 9. Assets
 10. PM records
-

7.3 No Fake Data Rule

Do not create fake dashboard or report outputs unless the prompt specifically asks for structured mock data for stress testing.

When mock data is used, it must be clearly tagged and governed.

7.4 Truth Classification Rule

All imported, normalized, or reported records must be classified honestly.

Use:

1. Fully resolved
2. Partially resolved
3. Unresolved

And / or:

1. Fully reportable
2. Partially reportable
3. Not reportable yet

No partial record should silently feed executive certainty.

7.5 Signal Proof Rule

Every signal must follow:

SIGNAL → CLICK → SUPPORTING RECORDS → DECISION

No dead counts.

No empty drill-down panels.

No unverified numbers.

7.6 Control Tower Rule

The Control Tower is the customer-facing validation source.

It must remain simple, industry-facing, and action-driven.

It should show:

1. Operational Health
 2. Work Order Performance
 3. Turn Performance
 4. PM Performance
 5. Asset Performance
 6. Priority Actions
-

7.7 Reports Rule

Reports must follow:

REAL DATA → SHARED LOGIC → REPORT STRUCTURE → EXPLAINABLE OUTPUT

No fabricated trends.

No generic storytelling.

No unsupported conclusions.

7.8 Import Governance Rule

Users may upload imperfect data.

The system must classify it honestly.

Flexible ingestion should preserve momentum.

Strict validation should protect wiring truth.

7.9 Language Separation Rule

Do not mix operational categories.

Work order language belongs to work orders.

Turn language belongs to turns.

PM language belongs to PM.

Asset language belongs to assets.

If the system says “turn impact” during a work order report, that is a language/wiring failure.

7.10 Audit Rule

After major builds, run a wiring audit or clickable signal verification.

A build is not truly complete just because the UI appears updated.

It is complete when the data proves the wiring.

8. Current Build Continuity Map

Current Active Stack

1. Foundation
2. Unit intake
3. Unit profile
4. Assignment engine
5. Workflow engine
6. Work order logic
7. Turn logic
8. Scoring engine
9. Alert engine
10. Dashboard intelligence
11. Document/evidence engine
12. Evidence visibility
13. Signal language
14. Reaction layer
15. Dual-mode import governance
16. Control Tower dashboard pre-wiring

17. Build 7 reporting spine
 18. Build 7.1 reporting ingestion
 19. Wiring audit system
 20. Clickable signal verification
-

Current Customer-Facing Direction

The system should move toward a clean self-service website/app experience where users can:

1. Upload reports
2. See what Ascent can reveal
3. Understand operational performance quickly
4. Drill into supporting records
5. Start a free trial
6. Continue using the same web-based system without going to a separate product

The website and app experience should feel like one connected system.

Current Control Tower Direction

The Control Tower is the main operational face.

It should answer:

1. Are we on track?
 2. What is the problem?
 3. What should we do next?
 4. What records prove it?
 5. What matters after that?
-

9. Future Build Handling

9.1 Before Writing a New Prompt

Before creating any new Replit prompt, check:

1. What system already exists?
2. Which prior layer does this build extend?
3. Does this duplicate existing logic?
4. Does this preserve the shared source of truth?
5. Does this protect partial vs confirmed data?
6. Does this preserve clickable proof?

7. Does this preserve Control Tower clarity?
 8. Does this require a wiring audit afterward?
-

9.2 After Writing a New Prompt

Every future prompt should include:

1. Source continuity note
 2. Build context
 3. What already exists
 4. Do not rebuild rules
 5. Core goal
 6. Specific build requirements
 7. Data rules
 8. Wiring rules
 9. Role rules
 10. Empty/low-data rules
 11. Performance rules
 12. Success state
 13. Important limits
 14. Internal label
 15. Required validation
-

9.3 After Running a Build

After Replit completes a build, review:

1. Build summary
2. What was built
3. What files changed
4. Screenshots
5. Whether UI matches the prompt
6. Whether the system actually wired the data
7. Whether dashboard counts are clickable
8. Whether reports distinguish partial vs confirmed data
9. Whether work order / turn / PM / asset language stayed clean
10. Whether the build needs audit or correction

Then update this Master Spine.

10. Final Source Strategy

This Master Source Index + Build Continuity Spine should now become the main reference file for Ascent 1.0.

It should be saved and uploaded as a source.

Once saved, older individual source files can be removed from active source slots if their logic is already captured here.

Going forward:

1. Keep this Master Spine active.
2. Keep only the most important active authority files uploaded.
3. Archive old build prompts into this document.
4. Use the Source Memory Header in every future Replit prompt.
5. Update this file after every major build.
6. Validate with wiring audits before trusting the system.

The scattered file stack becomes one controlled source system.

That is how Ascent 1.0 keeps moving without losing its spine.